Topologia nativa de rede (PostGIS Topology)

Este documento descreve **o que** o modelo entrega ao produto TabGis e **como** está construído tecnicamente sobre a extensão oficial `postgis_topology`.

---

1. Perspectiva funcional

Objetivo

Representar **segmentos lineares da rede** (conductores, tubagens, cabos, etc.) como linhas que **partilham vértices de forma consistente**: quando dois segmentos coincidem num ponto dentro da **tolerância** definida para a topologia, o PostGIS garante o **mesmo nó topológico**. Isso evita "pontos duplicados" que não se ligam entre si em análises de continuidade.

O que o operador ou integrador ganha

- **Continuidade explícita**: primitivas `node` / `edge_data` no schema da topologia (`tabgis_utilities_topo`) reflectem a malha administrada pelo motor topológico.
- **Dupla representação legível**:
  - `span_geom` (`geometry(LineString,4326)`): interoperável (GeoJSON na API, WFS no futuro).
  - `topo_geom` (`topology.TopoGeometry`): referência à topologia nativa, adequada a operações que respeitam nós partilhados.
- **Pontos de ligação ao cadastro**: `equipment_from_id` / `equipment_to_id` opcionais para associar segmentos a `gis.equipment` quando fizer sentido de negócio.

O que ainda não é (marcos futuros)

- **Edição transacional via API REST** destes segmentos (esta entrega é modelo + leitura).
- **Camada de mapa dedicada** no Leaflet (pode ser acrescentada reutilizando `GET /api/topology/spans`).
- **CRS projetado** para tolerância em metros constantes em toda a área — ver secção 5.

---

2. Perspectiva técnica

Extensão e topologia nomeada

1. `CREATE EXTENSION postgis_topology` — carrega funções e catálogos globais (`topology.topology`, `topology.layer`, …).
2. `topology.CreateTopology(name, srid, precision)` — cria um **schema** com o nome da topologia (ex.: `tabgis_utilities_topo`) contendo as tabelas internas:
   - `edge_data` — arestas dirigidas;
   - `node` — vértices;
   - `face` — faces (úteis se mais tarde existirem polígonos de zona ou subdivisão planar);
   - `relation` — associação entre feições e primitivas.

Parâmetros TabGis:

Parâmetro	Valor actual	Notas
Nome	`tabgis_utilities_topo`	Constante acordada no código (`backend/server.js`) e na migração SQL.
SRID	**4326**	Alinhado a `gis.equipment.geom` / restantes camadas utilities em graus.

Parâmetro	Valor actual	Notas
`precision`	**1e-6** graus	Ordem de grandeza ~cm–dm conforme latitude; afinar se necessário.

Camada de TopoGeometry

`topology.AddTopoGeometryColumn(topology, schema, table, column, type)` regista uma camada e adiciona a coluna. Para segmentos lineares usamos tipo **`LINE`** (equivalente a LineString na camada).

População: `toTopoGeom`

`topology.toTopoGeom(geom, topology_name, layer_id, tolerance)` converte a geometria simples em `TopoGeometry` e **materializa** primitivas na topologia. A tolerância efectiva utilizada na migração é a **`precision`** registada em `topology.topology` para esta topologia (compatível com a versão PostGIS em uso — valores negativos como "usar padrão" não são aceites em todas as builds).

Artefactos de negócio em `gis`

Objeto	Função
`gis.network_span`	Um segmento linear com `span_geom` obrigatório e `topo_geom` preenchido após conversão.
`gis.v_network_span_readable`	Vista com comprimento geodésico aproximado (`length_m_approx`) para leitura humana.

Migração versionada

Ficheiro: **`infra/sql/21_postgis_topology_utilities.sql`**

- Idempotente na medida do possível (`CREATE IF NOT EXISTS`, `ON CONFLICT DO NOTHING` nos demos).
- Inclui **três segmentos de demonstração** que partilham vértices — útil para validar fusão de nós.

---

3. API HTTP (leitura)

Requer o mesmo perfil que operações sobre ordens quando **`SECURITY_ENFORCE=1`**: privilégio **`module.oms`** e cabeçalho **`X-TabGis-User-Id`** válido.

Método	Rota	Resposta
`GET`	`/api/topology/meta`	Estado da topologia `tabgis_utilities_topo`, camadas em `topology.layer`, contagens de `edges` / `nodes` / `faces`. Se a topologia não existir, `{ ok, installed: false, hint }`.
`GET`	`/api/topology/spans?limit=`	Lista segmentos lineares com `span_geojson`, `has_topo_geom`, `length_m_approx` (máx. 500).

Sem enforcement de segurança (ambiente dev típico), as rotas respondem sem cabeçalho.

---

4. Operação e validação

Aplicar ou repetir a migração

```
docker exec -i infra-db-1 psql -U tabgis_admin -d tabgis_db -v ON_ERROR_STOP=1 \
  < infra/sql/21_postgis_topology_utilities.sql
```

Verificações rápidas em SQL

```
SELECT name, srid, precision FROM topology.topology WHERE name = 'tabgis_utilities_topo';

SELECT COUNT(*) AS edges FROM tabgis_utilities_topo.edge_data;
SELECT COUNT(*) AS nodes FROM tabgis_utilities_topo.node;

SELECT id, code, topo_geom IS NOT NULL AS has_topo FROM gis.network_span ORDER BY id;
```

Validar integridade topológica (opcional)

```
SELECT * FROM topology.ValidateTopology('tabgis_utilities_topo');
```

Corrigir erros reportados antes de confiar em análises de continuidade em produção.

---

5. Limitações e boas práticas

1. **Geografia vs geometria**: comprimentos na vista/API usam cast para **geografia** sobre 4326 — adequado para ordens de grandeza operacional; estudos de perfil eléctrico finos podem exigir modelo projetado.
2. **Tolerância em graus**: redes muito extensas podem preferir uma topologia separada em **EPSG projectivo local** com `precision` em metros.
3. **Redes não planares**: múltiplos circuitos sobrepostos podem exigir modelação avançada (várias camadas ou grafos além deste MVP).
4. **Rollback**: remover a topologia destrói primitivas — usar `DROP SCHEMA tabgis_utilities_topo CASCADE` apenas em consciência (impacto total nos `TopoGeometry` dependentes).

---

6. Referências cruzadas

Documento	Conteúdo relacionado
`docs/mds/CONTEXT_STATE.md`	Estado da stack, lista de scripts SQL, rotas da API.
`docs/mds/Database_Structure_ContextMod.md`	Modelo conceptual `gis` / utilities.
`docs/mds/DEPLOY.md`	Ordem de aplicação de migrações.

---

7. Regionalização (traduções EN / PT / ES / IT)

Textos de interface e glossário técnico **sem ambiguidade coloquial** estão cadastrados em **`infra/sql/02_populate_translations`** (tabela `config.translation`), com `ON CONFLICT DO UPDATE` para reaplicação segura:

Chave	Uso
`UI_GLOSSARY_LINEAR_NETWORK_SEGMENT`	Definição de *segmento linear da rede* (SIG / utilities).
`UI_GLOSSARY_POSTGIS_TOPOLOGY`	Definição da extensão `postgis_topology`.
`UI_HELP_STACK_POSTGIS_TOPOLOGY`	Linha de ajuda: stack PostGIS + `gis.network_span`.
`UI_HELP_API_TOPOLOGY`	Linha de ajuda: endpoints `/api/topology/*`.

Após alterar o ficheiro SQL, executar `psql` contra a base e **reiniciar** o contentor `api` para o cliente voltar a carregar o mapa de traduções.